FANTA AI · RESOCONTO TECNICO

Il nuovo backend

NestJS 12, Prisma 7 e Supabase: cosa è stato fatto, perché, e cosa viene dopo

16 ottobre 2026 · cartella `apps/api` del monorepo fanta-ai

IN BREVE

Il vecchio backend Express è stato sostituito da uno scheletro NestJS allineato alla roadmap: un monolite modulare in ESM, con Prisma 7 su Postgres e Supabase per database e autenticazione. Non contiene ancora logica di dominio, ma tutta l'infrastruttura che i moduli futuri useranno: verifica dei token di Supabase, errori in formato standard, log strutturati e correlabili, health check, validazione della configurazione e documentazione OpenAPI generata dal codice. Un unico endpoint applicativo, `GET /v1/me`, attraversa tutti gli strati e fa da modello. Tutto è stato provato su uno stack Supabase locale con un utente reale, oltre che con 18 test automatici.

STACK	ENDPOINT	TEST	STATO
NestJS 12 · Prisma 7.10 · Supabase	**/health/live · /health/ready · /v1/me**	**18 verdi, senza database**	**Codice Express rimosso, nessun commit**

1 Da dove siamo partiti

Il backend precedente (Express 5 e Prisma 6 su SQLite) nasceva da un'altra idea di prodotto: importare giocatori e statistiche da API-Football e lavorarli. Lo schema aveva una tabella per ogni blocco di statistiche (tiri, passaggi, contrasti, cartellini…) e gli endpoint coprivano l'import e una sola «mia squadra», globale e senza autenticazione.

Con la roadmap il centro del prodotto diventa il ranking settimanale prodotto da una pipeline AI centralizzata, con rose per utente, snapshot versionati e Supabase per dati e identità. Del vecchio codice non restava quasi nulla di riusabile, quindi l'ho rimosso invece di adattarlo: controller, route, servizi, repository, mapper, tipi di API-Football e la migration SQLite. Resta comunque recuperabile dalla cronologia di git.

2 Perché NestJS, e cosa resta di Prisma

NestJS era già la scelta della roadmap (ADR-002: monolite modulare NestJS con Supabase Postgres/Auth e Prisma) e, dovendo riscrivere tutto, cambiare framework costava poco o nulla in più. NestJS organizza il codice in moduli con dependency injection e offre meccanismi standard per gli aspetti trasversali: i guard per l'autorizzazione, le pipe per la validazione, i filter per gli errori, i middleware per i log. È ciò che serve a un monolite che dovrà ospitare catalogo, rose, ranking, formazioni e pipeline AI senza diventare un unico file. In più genera la specifica OpenAPI dai controller, che è la base del client mobile tipizzato previsto dalla roadmap.

Prisma è rimasto, come chiedevi, ma aggiornato alla versione 7 e spostato da SQLite a Postgres. La 7 cambia parecchio rispetto alla 6: il client viene generato come codice TypeScript dentro il progetto (`src/generated/prisma`, fuori da git), non c'è più il motore Rust per le query e la connessione passa da un driver adapter, qui `@prisma/adapter-pg` basato sul driver `pg`.

3 Le versioni scelte

Diverse dipendenze avevano appena cambiato versione major, quindi prima di scegliere ho controllato compatibilità e documentazione ufficiale. Non sempre l'ultima versione era quella giusta:

Componente	Versione	Perché
NestJS	12.1	I pacchetti core ora sono solo ESM: il progetto è `"type": "module"`, come il template ufficiale.
TypeScript	6.0	La 7, il nuovo compilatore nativo, non è ancora supportata da typescript-eslint.
Prisma	7.10	Il tag `latest` di npm punta a una pre-release di Prisma 8, che non ha `migrate` né `generate`.
Vitest	4.1 + Vite 8	È il default di NestJS 12 per i progetti ESM; i decorator funzionano senza compilatori aggiuntivi.
ESLint	10 + typescript-eslint	Regole che usano i tipi (ad esempio le promise non attese); Prettier con lo stile già in uso nel progetto.
Supabase CLI	2.121	Installata come dipendenza della root: nessuna installazione globale da ricordare.
pnpm	allowBuilds	Autorizzati gli script d'installazione di Supabase CLI e Prisma; bloccata la telemetria di swagger-ui.

Una conseguenza pratica dell'ESM: negli import relativi si scrive l'estensione `.js` anche quando il file sorgente è `.ts` (ad esempio `import { AppModule } from "./app.module.js"`). È la regola di Node per i moduli ES, non un errore.

4 Come è organizzato il codice

```
apps/api/
├── prisma/
│   ├── schema.prisma           modello profiles
│   └── migrations/             20261015152720_init
├── prisma.config.ts            URL per la CLI, caricamento del .env
├── src/
│   ├── main.ts                 avvio, logger, chiusura ordinata
│   ├── app.module.ts           moduli + guard, filtro errori, validazione globali
│   ├── app.setup.ts            request id, helmet, versioning /v1, Swagger
│   ├── config/                 variabili d'ambiente validate, logger pino
│   ├── common/                 problem details, request id
│   ├── prisma/                 PrismaService con adapter pg
│   ├── health/                 /health/live e /health/ready
│   ├── identity/               verifica dei token, AuthGuard, /v1/me
│   └── generated/prisma/       Prisma Client generato, fuori da git
└── test/                       test end-to-end
```

Ogni area di dominio sarà un modulo Nest: oggi c'è `identity`, poi arriveranno `catalog`, `squads`, `rankings`, `lineups`, `analysis`, `evaluation` e `operations`, come nel volume 3 della roadmap. Dentro un modulo il flusso è controller → service (lo use case) → Prisma: i controller non toccano mai il database. Guard, filtro degli errori

e validazione sono registrati una sola volta in `AppModule`, quindi valgono ovunque, test compresi. La configurazione HTTP che non appartiene a un modulo (id di richiesta, helmet, versioning, Swagger) sta in `app.setup.ts`, condivisa da `main.ts` e dai test end-to-end.

Il percorso di una richiesta

1. **Request id** — riusa l'`x-request-id` del client se ben formato, altrimenti genera un UUID
2. **Helmet** — aggiunge gli header di sicurezza HTTP
3. **Body parser** — legge il JSON, con un limite di 100 KB
4. **Log** — pino-http registra id, metodo, URL, utente ed esito
5. **AuthGuard** — richiede un token Supabase valido, salvo le route `@Public()`
6. **ValidationPipe** — valida i DTO e rifiuta i campi non dichiarati
7. **Controller → service → Prisma** — la logica applicativa e l'accesso ai dati
8. **ProblemDetailsFilter** — qualunque errore, in qualunque punto, diventa problem+json

5 Il database: Supabase locale, Prisma e lo schema app

In locale gira lo stack Supabase in Docker (Postgres 17, Auth, Studio e Mailpit per le email di prova), configurato in `supabase/config.toml` e avviato dalla CLI. Ho spento i servizi che il prodotto non usa ancora (realtime, storage, edge functions, analytics) e anche il seed di Supabase, che girerebbe prima delle migration di Prisma.

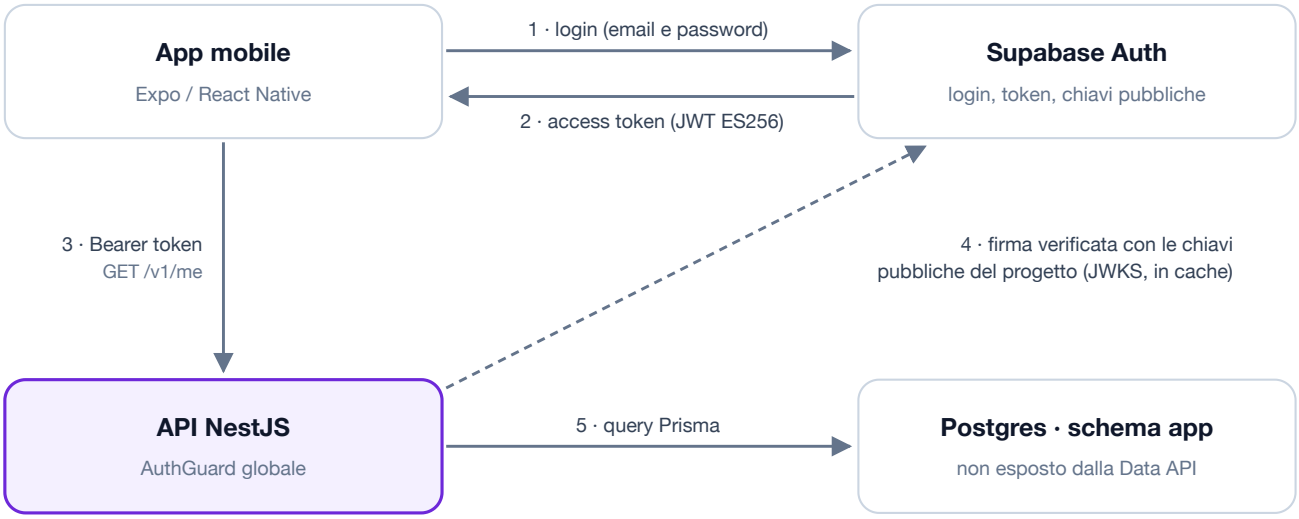
La decisione più importante riguarda dove vivono le tabelle. Supabase espone automaticamente lo schema `public` attraverso la sua Data API (PostgREST), raggiungibile con la chiave pubblica contenuta nell'app. Una tabella creata da Prisma in `public` sarebbe quindi leggibile e modificabile dal client, a meno di ricordarsi di attivare la RLS su ogni tabella, compresa quella interna di Prisma (`_prisma_migrations`). Ho messo invece tutto nello schema `app`, che la Data API non espone: il mobile non può interrogare i dati di dominio e passa sempre dall'API, come prevede la roadmap. L'ho verificato direttamente: PostgREST risponde che la tabella non esiste e che lo schema `app` non è tra quelli esposti.

Gli URL di connessione sono due: `DATABASE_URL`, usato dall'app a runtime (in cloud punterà al pooler Supavisor), e `DIRECT_URL`, usato dalla CLI di Prisma per migration e Studio; in locale coincidono. Lo schema è indicato con `?schema=app` e passato esplicitamente all'adapter, che non lo ricava dall'URL. La connessione al database è lazy: l'API parte anche se Postgres non è ancora pronto e lo segnala da `/health/ready`, invece di andare in crash all'avvio.

Per ora c'è una sola tabella, `profiles`: l'id coincide con quello dell'utente in Supabase Auth, più nome visualizzato, fuso orario (default `Europe/Rome`) e timestamp. La migration iniziale è `20261015152720_init`, e ho verificato che `migrate deploy` ricrei lo schema da zero, che è lo scenario di staging e produzione. Catalogo, rose e giornate li ho lasciati a te, come suggerisce la roadmap: prima entità e invarianti, poi le migration.

> Prisma 7 non rigenera più il client dopo `migrate dev`: lo fa lo script `db:migrate`. Il client si rigenera anche a ogni `pnpm install`, senza bisogno di un database.

6 L'autenticazione: Supabase autentica, l'API autorizza



Il login resta in Supabase (1–2); l'API controlla il token con le chiavi pubbliche del progetto (4) e solo dopo accede ai dati (5).

Il login resta in Supabase Auth, che rilascia un access token JWT. L'app lo invia all'API nell'header `Authorization: Bearer` e un guard globale lo verifica a ogni richiesta con la libreria jose: firma, emittente (issuer), destinatario (audience `authenticated`) e scadenza. Il guard è attivo per default su tutte le route e quelle pubbliche vanno dichiarate con `@Public()`: un endpoint nuovo non può nascere scoperto per dimenticanza. Nei controller l'utente verificato si legge con `@CurrentUser()`.

Le prove hanno chiarito un dettaglio importante: anche lo stack locale firma i token con chiavi asimmetriche (ES256), come i progetti cloud che usano le nuove signing key. L'API verifica quindi le firme con le chiavi pubbliche del JWKS del progetto, che jose scarica, tiene in cache e aggiorna quando Supabase le ruota: non serve alcun segreto condiviso. Il vecchio secret HS256 resta supportato solo se configurato (`SUPABASE_JWT_SECRET`), per un eventuale progetto legacy.

Il primo endpoint applicativo, `GET /v1/me`, restituisce il profilo dell'utente e lo crea al primo accesso con un upsert idempotente. L'autorizzazione più fine, cioè chi può leggere o modificare quale rosa, resterà negli use case dei singoli moduli.

> **Perché issuer e audience contano.** Durante le prove la anon key di Supabase, firmata con lo stesso secret dello stack locale, aveva una firma valida. È stata rifiutata solo perché non ha l'audience `authenticated` né l'issuer del progetto.

7 Errori, log e health check

Ogni errore esce nello stesso formato, `application/problem+json` (RFC 9457), con `type`, `title`, `status`, `detail`, il path della richiesta (`instance`) e un `traceId` uguale all'header `x-request-id`:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Access token mancante.",
  "instance": "/v1/me",
  "traceId": "mobile-req-123"
}
```

Se l'app invia il proprio `x-request-id`, l'API lo riusa: un errore visto sul telefono si ritrova nei log del server con lo stesso id. Gli errori di validazione elencano i vincoli violati nel campo `errors`. Quelli imprevisti diventano un 500 generico: stack e messaggi interni finiscono solo nei log, mai nella risposta. Per gli errori di dominio con un significato stabile, come il conflitto di versione della rosa descritto nella roadmap, c'è `ProblemDetailsException`, che consente un `type` dedicato e campi aggiuntivi come `currentVersion`.

I log sono JSON strutturati con pino, resi leggibili da pino-pretty in sviluppo. Per ogni richiesta registrano id, metodo, URL, id utente, status e durata, ma non gli header (quindi nessun token) né gli indirizzi IP, che sono dati personali. Le chiamate ai probe `/health/*` non vengono registrate, per non fare rumore.

Gli health check sono due e stanno fuori dal versioning: `/health/live` dice solo che il processo risponde (liveness), `/health/ready` controlla anche il database con un `SELECT 1` e risponde 503 se non è raggiungibile (readiness). Alla ricezione di SIGTERM, il segnale che Railway invia durante i deploy, l'app si chiude in modo ordinato: smette di accettare richieste, completa quelle in corso e chiude le connessioni al database.

8 Configurazione e documentazione dell'API

Le variabili d'ambiente sono validate all'avvio in `src/config/env.ts`: se manca `DATABASE_URL` o `SUPABASE_URL`, o un valore non è valido, l'app non parte e spiega perché. Il file `.env.example` contiene solo i valori dello stack locale, mentre `.env` resta fuori da git.

La documentazione OpenAPI è generata dal codice. Il plugin di `@nestjs/swagger` legge i tipi TypeScript dei DTO e i commenti JSDoc, che diventano descrizioni e summary senza decorator da ripetere. In sviluppo la Swagger UI è su `/docs` e il documento JSON su `/docs/openapi.json`: sarà la sorgente da cui generare il client mobile tipizzato. In produzione `/docs` non viene esposto.

9 Come è stato verificato

Oltre a lint, typecheck e build, ci sono 18 test con Vitest. Nessuno richiede un database, quindi girano ovunque, CI compresa:

Area	Cosa verificano
Verifica dei token · 7 unit	Token ES256 valido con un JWKS simulato; rifiuto di token scaduti, emessi da un altro progetto o senza audience `authenticated`; HS256 rifiutato senza secret e accettato solo con il secret giusto.
Filtro errori · 3 unit	Nessun dettaglio interno nei 500; errori di validazione in `errors`; tipo ed estensioni dei problemi di dominio.
HTTP end-to-end · 8 e2e	Health pubblico; 401 in problem+json con `traceId`; riuso dell'`x-request-id`; 404; JSON malformato → 400; body troppo grande → 413; documento OpenAPI.

Poi la prova sul campo, sullo stack Supabase locale e con un utente reale registrato tramite Supabase Auth:

- migration applicata, anche con `migrate deploy` su uno schema vuoto;
- `/v1/me` con il token ottenuto: 200, e il profilo viene creato una sola volta anche chiamando più volte;
- anon key usata come token: rifiutata;
- tabella `profiles` invisibile alla Data API di Supabase;
- `/health/ready` a 200 con il database e a 503 senza;
- build di produzione con Swagger disattivato e header di sicurezza attivi;
- chiusura ordinata su SIGTERM, anche con connessioni keep-alive aperte.

Queste prove hanno fatto emergere quattro problemi, tutti corretti e, dove possibile, coperti da test:

- un body oltre i 100 KB rispondeva 500 invece di 413;
- gli errori sollevati durante la lettura del body uscivano senza `traceId`, perché l'id veniva assegnato dopo: ora lo assegna il primo middleware della catena;
- il 503 volontario della readiness veniva registrato due volte, anche come errore con stack trace;
- le righe di log delle richieste includevano tutti gli header e l'IP del client.

10 Come si usa

```
pnpm install        # dipendenze e Prisma Client
pnpm db:start       # stack Supabase locale (richiede Docker)
pnpm db:migrate     # applica le migration allo schema app
pnpm api            # API in watch mode
```

Dalla root sono disponibili anche `api:test`, `api:lint`, `api:typecheck` e `api:build` per l'API, e `db:status`, `db:stop`, `db:reset` e `db:studio` per il database. Supabase Studio è su `http://127.0.0.1:54323`, la documentazione dell'API su `/docs`. Sulla tua macchina la porta 3000 è già occupata da un altro progetto (un server Next.js): imposta `PORT` in `apps/api/.env`.

11 Cose da sapere

- La CLI di Prisma segnala un aggiornamento a 8.0.0-rc: va ignorato finché Prisma 8 non è stabile.
- `pnpm db:reset` funziona, ma Prisma 7 blocca `migrate reset` se lo lancia un agente AI senza il consenso esplicito dell'utente: per questo non l'ho eseguito.
- Il seed dei dati di dominio, come i 20 giocatori fittizi della W2, va fatto con Prisma e non con `supabase/seed.sql`.
- Docker Desktop era spento e l'ho avviato per le prove: lo stack Supabase ora è fermo (i dati restano nei volumi), Docker è ancora aperto.
- Rispetto alla checklist della W1 ho anticipato voci della W2–W3: bootstrap di NestJS, guard JWT, OpenAPI, problem details, health check, Supabase locale.
- Il nuovo lockfile non cambia le versioni delle dipendenze del mobile.
- Non ho fatto commit: tutte le modifiche sono nel working tree, pronte da rivedere.

12 Prossimi passi

Quando	Cosa
W2 · Schema di dominio	Catalogo (squadre, giocatori, ruoli, calendario, giornate) e rose versionate, partendo da entità e invarianti; seed con giocatori fittizi tramite Prisma.
W3 · CI e container	GitHub Actions con lint, typecheck, test e migration su un Postgres di servizio; Dockerfile multi-stage per API e worker.
W5 · Contratto col mobile	Export di `openapi.json` in CI e `packages/api-client` generato con openapi-typescript e openapi-fetch; test di integrazione con due utenti per l'ownership delle rose.
W14 · Staging	Progetto Supabase cloud, `DATABASE_URL` sul pooler, utente Postgres dedicato a Prisma, rate limit sugli endpoint costosi come la generazione della formazione.

Il backend oggi è piccolo ma completo: ogni richiesta è autenticata, tracciabile e con errori prevedibili. Da qui in avanti le nuove feature dovrebbero aggiungere moduli, non infrastruttura.